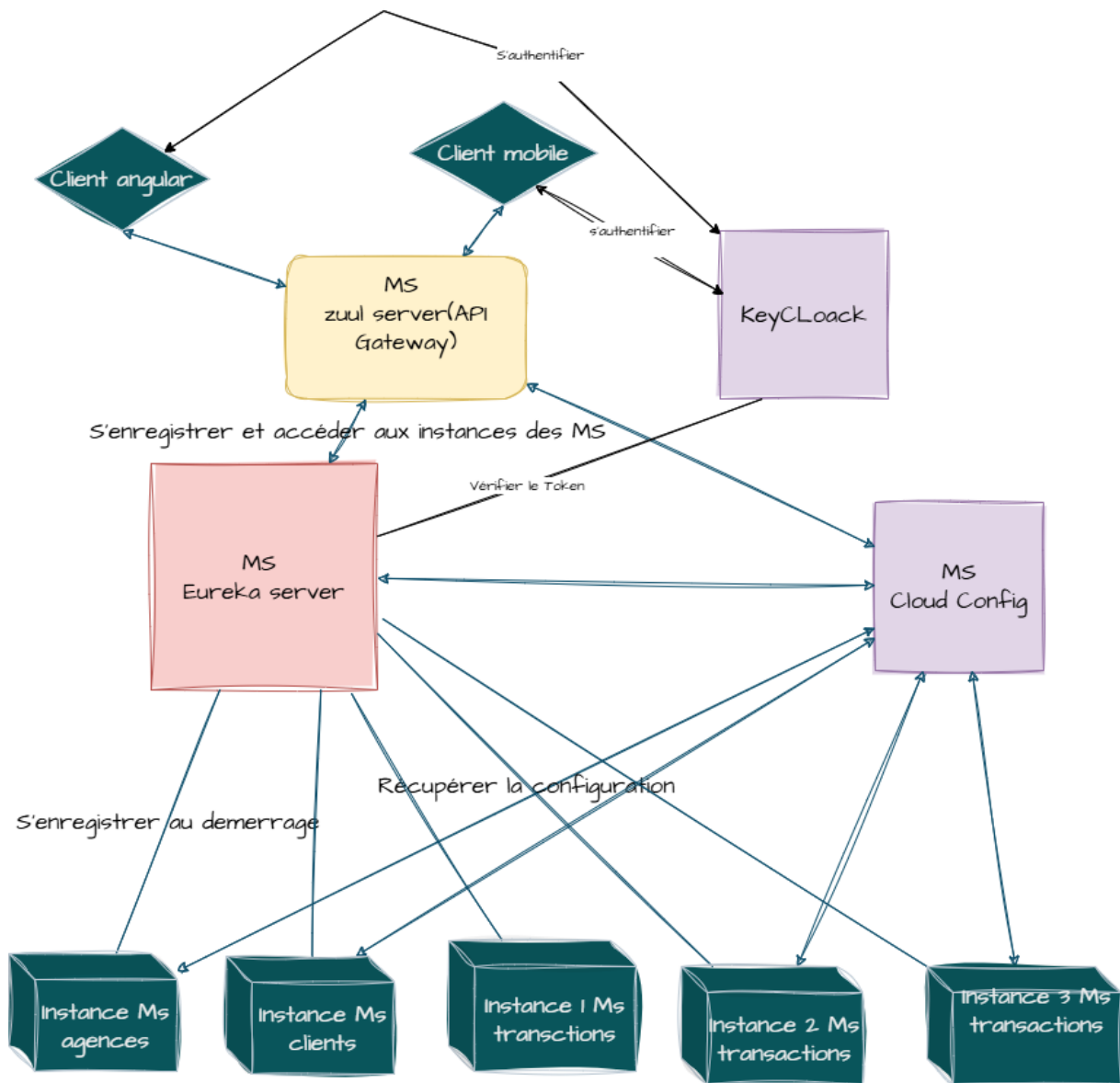


ITMA-STRI-S3

Travaux pratiques Architecture à Base de Composants Logiciels (Gestion d'une banque)

Soit l'Architecture Orientée MicroServices suivante :



Nous souhaitons utiliser une architecture micro service pour la gestion d'une banque.

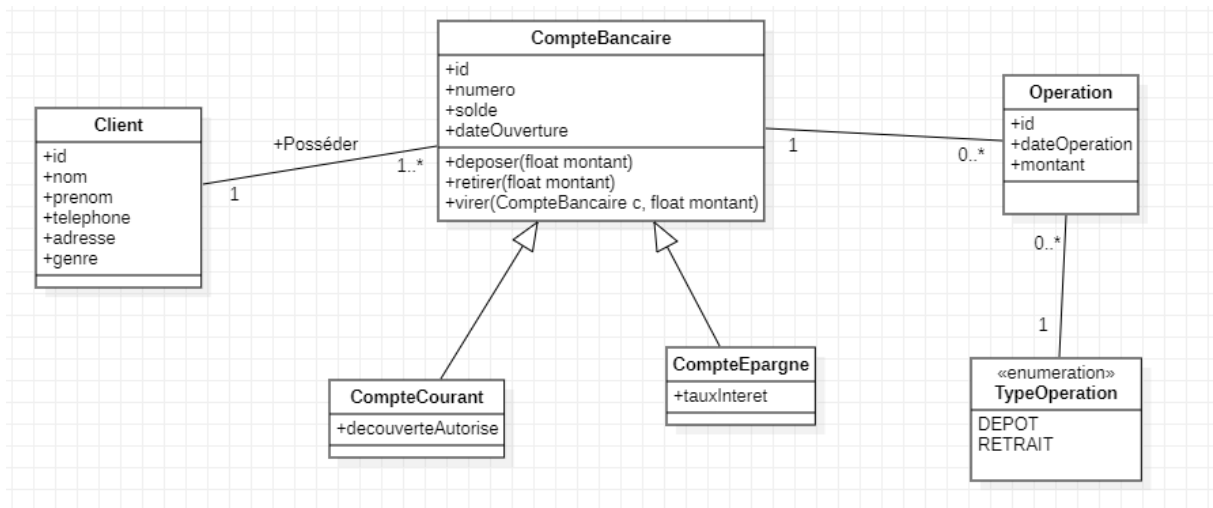
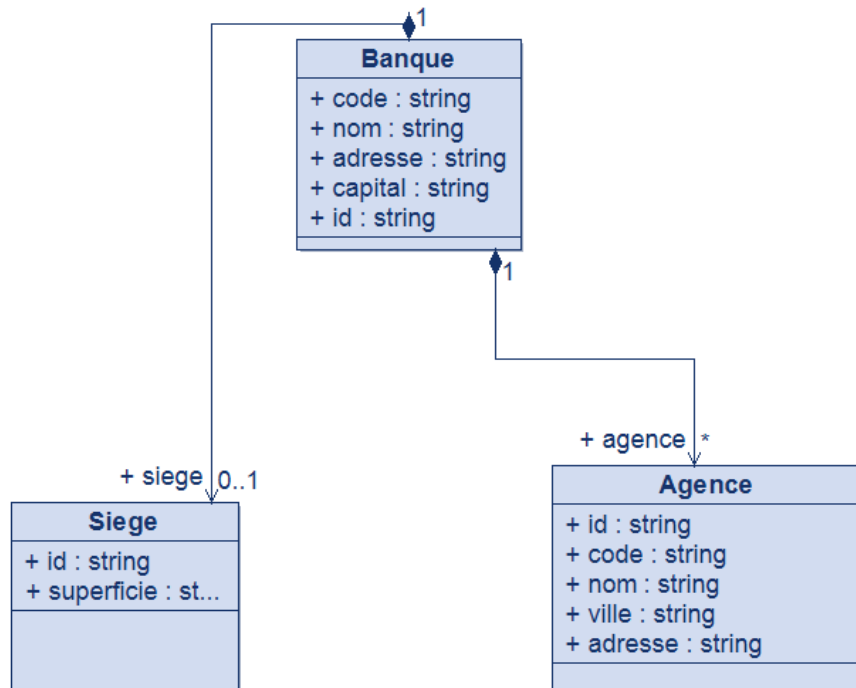
Les micros services seront les suivants :

1. ms-agences : Qui gère la structure de la banque : le siege et ses agences
2. ms-clients : qui gère les clients de la banque et leur compte
3. ms-transactions : qui gère les opérations bancaires (dépôt, retrait,..)

Proposer une modelisation sur la base du diagramme de classes suivante pour gérer ce système, en ajoutant les classes :Banque, Agence, Siege. Une banque est composée d'un seul siege et d'une ou plusieurs agences.

Nous voulons que chaque MS ait sa propre base de données et replique les données d'autres ms selon les besoins.

Ms-agences



Exercice 1 : Création du projet ms-agences

1. Créer un projet spring boot nommé **ms-agences** [Spring Initializr](#) avec les paramètres suivants :
 - a) Project : maven
 - b) Language : Java
 - c) Spring boot : 3.2.2

- d) Group : ml.itma
 - e) Artific : ms-agences
 - f) Packaging : Jar
 - g) Java : 20 ou 17
 - h) Dependencies : **JPA, Web, Lombok, MySQL Driver**
2. Cliquer sur GENERATE afin de générer le projet.
 3. Décompresser l'archive téléchargé dans votre espace de travail et l'importer dans Eclipse.
 4. Se positionner dans la racine du projet et taper **cmd** dans la barre d'adresse de l'explorateur de fichiers.
 5. Dans l'invité de commande, taper **mvn package** (cette commande va compiler le projet et télécharger les dépendances choisies)

Exercice 2 : Configuration du projet

1. Créer dans votre SGBD (Mysql ou PostgreSQL), une base de données nommée « banque_agence »
2. Créer un utilisateur dans cette base ayant tous les droits sur « bd_api »
3. Dans le fichier **src/main/resources/applications.properties** ajouter les lignes suivantes afin de lier notre projet à la base de données créée :

```
server.port=8085
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/bd_api
spring.datasource.username=user1
spring.datasource.password=user1
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
logging.level.org.hibernate.SQL=off
spring.profiles.active=dev
server.error.whitelabel.enabled=false
spring.messages.basename=messages
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ma_base
spring.datasource.username=postgres
spring.datasource.password=password

# Driver PostgreSQL
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

4. Dans le package principal, ajouter les packages suivants:
 - a) model
 - b) repository
 - c) service
 - d) controller

Exercice 3 : Mise en place du modèle

- a) Créer les classes modèles obtenues à partir de votre diagramme de classes.

Exercice 4 : Création des repository

- Créer un repository pour l'ensemble des entités définies.

Exercice 5 : Création des services

- Créer les services suivants pour avoir un CRUD complet par model.

Exercice 6 : Création des contrôleurs

- Créer un Controller pour exposer les différents services

Exercice 7 : Répéter les étapes de 1 à 6 pour implémenter les ms-clients, ms-transactions

Exercice 8

Intégration config-server

1. Créer un repository sur github « tp-arch-repository »
2. Créer une clé d'accès sur github, dans la partie developer
3. Créer le projet config-server (avec la dépendance spring-config-server), activer la configuration dans la classe principale du projet (@EnableConfigServer)
4. Dans le fichier application.properties ajouter les lignes suivantes

```
spring.cloud.compatibility-verifier.enabled=false
spring.application.name=config-server
server.port:9101
spring.cloud.config.server.git.uri=https://github.com/votre-username/nom-repository.git
spring.cloud.config.server.git.username=${GIT_USERNAME}
spring.cloud.config.server.git.password=votre_token
```

5. créer un dossier config-repos dans le repertoire principal (près ms-communication,ms-metier)
6. Dans ce dossier (config-repos), copier et coller application.properties de eureka. Renommer ce fichier eureka-server.properties
7. Initialisez un nouveau dépôt GIT local que nous allons pousser plus tard vers le distant : **git init**
8. Ajoutez les fichiers du dossier au GIT : **git add** .

9. Ajoutez l'URL du dépôt distant à celui-ci : **git remote add origin** <https://NOM-UTILISATEUR:MOT-DE-PASSE@github.com/lien-repository.git> Veuillez à faire les remplacements de nom d'utilisateur et de mot de passe, ainsi qu'à mettre votre propre URL à la place de la mienne.
10. Faites un commit du contenu : **git commit -m "Premier commit"**
11. Faites le push: **git push -u origin master**
12. Dans eureka-server, ajouter les dépendances suivantes et faites mvn package

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bootstrap</artifactId>
</dependency>
```

13. Renommer application.properties de eureka en bootstrap.properties et remplacer le contenu par :

```
spring.application.name=eureka-server
spring.cloud.config.uri=http://localhost:9101
management.endpoints.web.exposure.include=refresh
```

14. Démarrer config-server
15. Puis démarrer eureka-server
- 16.

Exercice 9 : Interface web angular

1. Créer le projet angular **ng new mybank-web-app**
2. Installer Bootstrap **npm install bootstrap** et **npm install bootstrap-icons**
3. Configurer **Bootstrap** dans **angular.json** ajouter les lignes suivantes

```
"styles": [
```

```

        "node_modules/bootstrap/dist/css/bootstrap.min.css",
        "node_modules/bootstrap-icons/font/bootstrap-icons.scss",
        "src/styles.css"
    ],

```

```

"scripts": ["node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"],

```

4. Créer les différents composants (menu-principal, nouveau-client, depot, retrait, solde, releve) **ng generate component nouveau-client**
5. Dans le composant menu-principal.component.html ajouter les lignes suivantes :

```

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">My-Bank</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarSupportedContent" aria-
controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarSupportedContent">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item">
          <a class="nav-link" aria-current="page" routerLinkActive="active"
routerLink="/">Accueil</a>
        </li>
        <li class="nav-item" >
          <a class="nav-link" routerLinkActive="active"
routerLink="/nouveau-client">Nouveau client</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" routerLinkActive="active"
routerLink="/solde">Consultation solde</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" routerLink="/depot"
routerLinkActive="active">Depôt</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" routerLink="/retrait"
routerLinkActive="active">Retrait</a>
        </li>
      </ul>
      <form class="d-flex">

```

```

        <input class="form-control me-2" type="search" placeholder="Search"
        aria-label="Search">
        <button class="btn btn-outline-success"
        type="submit">Search</button>
    </form>
</div>
</div>
</nav>

```

6. Nous allons afficher les informations d'un compte en utilisant un composant detail-compte, créer ce composant.
7. Créer les classes modèles suivants dans un dossier nommé models

```

export class Client {
    numero!:number;
    nom!:string;
    prenom!:string;
    adresse!:string;
    telephone!:string;
    genre!:string;
}

```

```

import { Client } from "../client";

export class CompteBancaire {
    id!:number
    numero!:number;
    solde!:number
    dateOuverture!:string;
    titulaire!: Client
}

```

```

export class Operation {
    id!:number;
    montant!:number;
    dateOperation!:string;
    typeOperation!:string;
}

```

8. Dans le fichier html du composant detail-compte ajouter le code html suivant :

```

@if (compte !=null) {
<div class="container">
    <div class="row justify-content-center">
        <div class="col-md-6">

```

```

        <div class="card">
            <div class="card-header bg-dark center-contenu text-
white">Informations du compte</div>
            <div class="card-body">
                <table class="table">

                    <tr>
                        <th scope="col">ID</th>
                        <th scope="row">{{ compte.id }}</th>
                    </tr>
                    <tr>
                        <th scope="col">Numéro</th>
                        <th scope="row">{{ compte.numero }}</th>
                    </tr>
                    <tr>
                        <th scope="col">Titulaire</th>
                        <td>{{ compte.titulaire.prenom }} {{
compte.titulaire.nom }}</td>
                    </tr>
                    <tr>
                        <th scope="col">Solde</th>
                        <th>{{ compte.solde }} FCFA</th>
                    </tr>
                    <tr>

                        <th scope="col">Date d'ouverture</th>

                        <td>{{ compte.dateOuverture }}</td>
                    </tr>

                </table>
                <a class="btn btn-primary" [routerLink]="['/releve-
compte', compte.numero]">Consulter le relevé</a>
            </div>
        </div>
    </div>
</div>
</div>
</div>
}
@else if(error && numero!=null ){
<br /><br />
<div class="alert alert-danger" role="alert">
    Ce numéro {{numero}} de compte n'existe pas!
</div>
}

```


9. Dans le fichier TypeScript du composant detail-compte ajouter les lignes suivantes :

```
import { Component, Input, OnInit } from '@angular/core';
import { CompteBancaire } from '../models/compte-bancaire';
import { OperationsService } from '../services/operations.service';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { CommonService } from '../services/common.service';
import { Subscription } from 'rxjs';

@Component({
  selector: 'app-detail-compte',
  standalone: true,
  imports: [RouterLink],
  templateUrl: './detail-compte.component.html',
  styleUrls: ['./detail-compte.component.css']
})
export class DetailCompteComponent implements OnInit {
  @Input() numero!:any;
  compte!:CompteBancaire|undefined;
  error:boolean = false;
  private subscriptionName!: Subscription;

  constructor(
    private commonService: CommonService,
    private operationService: OperationsService,
    private route: ActivatedRoute
  ){

    this.subscriptionName= this.commonService.getUpdate().subscribe
      (message => {
        this.numero = message.text;
        this.consulter();
      });

  }
  ngOnInit(): void {

    this.numero = this.route.snapshot.params['numero'];
    this.consulter();

  }

  consulter(){
    this.error=false;
    this.operationService.consulter(this.numero).subscribe(
      (c:CompteBancaire |undefined) =>{
        if(c){
          this.compte= c;
        }
      }
    );
  }
}
```

```

        }else{
            this.error=true;
            this.compte= undefined;
        }

    }

    );
}

ngOnDestroy() { // It's a good practice to unsubscribe to ensure no
memory leaks
    this.subscriptionName.unsubscribe();
}
}

```

10 . Ajouter les services suivants dans un dossier nommé services (ng g s nom)

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable, catchError, map, of } from 'rxjs';
import { CompteBancaire } from '../models/compte-bancaire';
import { Operation } from '../models/operation';

@Injectable({
    providedIn: 'root'
})
export class OperationsService {
    constructor(private http: HttpClient) { }

    // Méthode pour effectuer une requête POST
    operation(data:any): Observable<any> {
        if(data.type === "DEPOT")
            return this.deposer(data);
        else if(data.type === "RETRAIT")
            return this.retirer(data);
        else
            return this.virer(data);
    }

    depoter(data:any): Observable<any> {
        const url =
`http://localhost:8085/api/v1/operation/depot?numero=${data.numero}&montant=${
data.montant}`;
        return this.http.put<any>(url, {});
    }

    retirer(data:any): Observable<any> {
        const url =
`http://localhost:8085/api/v1/operation/retrait?numero=${data.numero}&montant=
${data.montant}`;

```

```

        return this.http.put<any>(url, {});
    }

    virer(data:any): Observable<any> {
        const url =
`http://localhost:8085/api/v1/operation/virement?numeroSource=${data.numero}&n
umeroDest=${data.numeroDest}&montant=${data.montant}`;
        return this.http.put<any>(url, {});
    }

    consulter(numero:number): Observable<CompteBancaire | undefined> {
        const url =
`http://localhost:8085/api/v1/operation/solde?numero=${numero}`;

        return this.http.get<CompteBancaire>(url).pipe(
            map((response: CompteBancaire) => response),
            catchError(() => of(undefined)) // Retourne undefined si une erreur se
produit
        );
    }

    getReleve(numero:number): Observable<Operation[] | undefined> {
        const url =
`http://localhost:8085/api/v1/operation/releve?numero=${numero}`;

        return this.http.get<Operation[]>(url).pipe(
            map((response: Operation[]) => response),
            catchError(() => of(undefined)) // Retourne undefined si une erreur se
produit
        );
    }
}

```

Classe CommonService

```

import { Injectable } from '@angular/core';
import { Observable, Subject } from 'rxjs';

@Injectable({
    providedIn: 'root'
})
export class CommonService {

    private numero = new Subject<any>(); //need to create a subject

    sendUpdate(message: string) { //the component that wants to update
something, calls this fn

```

```

        this.numero.next({ text: message }); //next() will feed the value
in Subject
    }

    getUpdate(): Observable<any> { //the receiver component calls this
function
        return this.numero.asObservable(); //it returns as an observable
to which the receiver funtion will subscribe
    }
}

```

10. Supprimer les composants depot, retrait et créer le composant operation qui va nous permettre de faire des dépôts et retrait dans un compte.
Ajouter ce code dans le fichier html du composant operation :

```

<div class="container">
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card">
        <div class="card-header bg-dark center-contenu text-
white">Nouvelle opération</div>
        <div class="card-body">
          <form (ngSubmit)="onSubmit()">
            <div class="form-group">
              <label for="numero">Type d'opération</label>
              <select id="numero" class="form-control"
[(ngModel)]="model.type" name="type">
                <option value="DEPOT">Depôt</option>
                <option value="RETRAIT">Retrait</option>
              </select>
            </div>
            <div class="form-group">
              <label for="numero">Numéro de compte</label>
              <input type="number" class="form-control"
name="numero" id="numero"
[(ngModel)]="model.numero"
(blur)="rafraichirDetailsCompte()">
            </div>
            <div class="form-group">
              <label for="montant">Montant</label>
              <input type="number" class="form-control"
id="montant" name="montant"
[(ngModel)]="model.montant"
(ngModelChange)="rafraichirDetailsCompte()">
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

                <br />
                <button type="submit" class="btn btn-
primary">Valider</button>
            </form>
        </div>
    </div>
</div>
</div>
</div>
<br />
<app-detail-compte (numero)="model.numero"></app-detail-compte>

```

TypeScript de ce composant :

```

import { CommonModule } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { FormsModule, Validators } from '@angular/forms';

import { OperationsService } from '../services/operations.service';
import { Router } from '@angular/router';
import { NONE_TYPE } from '@angular/compiler';
import { SoldeComponent } from '../solde/solde.component';
import { DetailCompteComponent } from '../detail-compte/detail-
compte.component';
import { CommonService } from '../services/common.service';

@Component({
  selector: 'app-depot',
  standalone: true,
  imports: [FormsModule, SoldeComponent, DetailCompteComponent],
  templateUrl: './depot.component.html',
  styleUrls: ['./depot.component.css']
})
export class OperationComponent implements OnInit {

  model = { type:'', numero: '', montant: '', numeroDest:'' };
  refresh: boolean = false;

  constructor(
    private commonService : CommonService,
    private operationService: OperationsService,
    private router: Router) {

  }
}

```

```

ngOnInit(): void {

}

onSubmit() {

    this.operationService.operation(this.model).subscribe(
        res=>{
            this.router.navigate(["/detail-compte/"+this.model.numero]);

        }
    );

}

rafraichirDetailsCompte() {
    this.commonService.sendUpdate(this.model.numero);
}

}

```

Complément du Backend

Préparer du côté du backend, un service qui retourne un compte bancaire à travers son numero, dans la classe OperationController en ajoutant les lignes

```

@GetMapping("solde")
    public CompteBancaire consulter(@RequestParam int numero)
        throws CompteNotFoundException {
        return operationService.consulter(numero);
    }

```

Dans OperationService

```

public CompteBancaire consulter(int numero) throws
CompteNotFoundException {
    CompteBancaire compte=compteBancaireRepository
        .findByNumero(numero);

    if(compte==null)
        throw new CompteNotFoundException(numero);
    return compte;
}

```